Laboratorul 2 · Comunicații cu ESP32

Sisteme dedicate pentru IoT

Îndrumător practic pentru studenți • 6 octombrie 2026

Cinci sarcini: conectare Wi-Fi, server web cu LED, Bluetooth Classic, notificări WhatsApp și e-mail cu text, HTML și atașamente.

Materialul dezvoltă laboratorul 2 din îndrumătorul furnizat de cadrul didactic. Explicațiile și schemele sunt redactate special pentru această lucrare, în limba română; nu reprezintă o traducere integrală a articolelor externe. Tutorialele indicate sunt păstrate în bibliografie pentru consultarea imaginilor și explicațiilor originale.

Adaptări aplicate: datele rețelei laboratorului, completate local în secrets.h; controlul LED-ului existent pe GPIO 2, fără LED-uri, rezistoare sau breadboard externe. Fișierul original „WI-FI connection test” menționat în directorul Files/Laboaratoare/ESP32 nu a fost furnizat; taskul 1 include un program echivalent, complet și explicat.

Materiale livrate: PDF cu cuprins clicabil, document Word editabil și arhivă cu cinci proiecte Arduino independente. Pentru taskul 5 sunt incluse un fișier TXT și o imagine PNG de test.

Verificare: codul a fost inspectat și confruntat cu API-urile documentate. Nu a fost compilat sau rulat pe o placă ESP32 în acest mediu. Confirmarea practică se face prin probele indicate la fiecare task.

Înaintea orei: verificați existența unei chei CallMeBot deja activate și a unui cont e-mail de laborator cu parolă de aplicație. La consultarea din 06.10.2026, CallMeBot anunța că botul este plin și nu afișa numărul pentru activări noi. Taskul 4 are și un mod local de simulare, care nu echivalează cu livrarea unui mesaj. [S5]

Cuprins

0. Pregătirea laboratorului 3

1. Task 1 · Testarea conexiunii Wi-Fi 5

2. Task 2 · Server web pentru LED-ul de pe GPIO 2 7

3. Task 3 · Bluetooth Classic și terminal serial 10

4. Task 4 · Mesaj WhatsApp de la ESP32 13

5. Task 5 · E-mail prin SMTP: text, HTML și atașamente 16

6. Fișa de rezultate și întrebări de verificare 21

7. Surse și corespondența cu îndrumătorul inițial 22

0. Pregătirea laboratorului

Echipamente și aplicații

• Placă cu ESP32 clasic, de exemplu ESP32-WROOM-32 / DOIT ESP32 DEVKIT V1; cablu USB de date și calculator. Taskul Bluetooth SPP nu este aplicabil direct plăcilor ESP32-S2, S3 sau C3. [S7]

• Telefon Android pentru terminalul Bluetooth; browser pe calculator sau telefon pentru serverul web. Pentru notificările prin Internet, sunt necesare accesul la serviciile respective și propriile credențiale.

• Arduino IDE 2.x; pentru încărcarea LittleFS, IDE 2.2.1 sau mai nou și extensia LittleFS Uploader. Biblioteca ReadyMail de Mobizt este necesară numai la taskul 5; API-ul verificat aici este cel din versiunea 0.4.2. [S8, S10]

Configurarea Arduino IDE

1. Conectați placa prin USB. În Boards Manager instalați „esp32 by Espressif Systems”, dacă nu există deja. Dacă este necesar un URL suplimentar pentru platformă, folosiți adresa oficială din sursa S11.

2. Selectați modelul corespunzător în Tools → Board: DOIT ESP32 DEVKIT V1 sau ESP32 Dev Module, în funcție de placa disponibilă. Selectați portul COM în Tools → Port.

3. Deschideți un singur proiect din arhivă. Fișierul .ino și directorul său au același nume. Nu lipiți cele cinci programe în același sketch: fiecare are propriile funcții setup() și loop().

4. Apăsați Verify, apoi Upload. Dacă încărcarea rămâne la „Connecting”, țineți BOOT apăsat până începe scrierea; eliberați apoi butonul. Deschideți Serial Monitor la 115200 baud și apăsați EN dacă doriți reluarea programului.

5. Înlocuiți G01 cu identificatorul echipei. Notați versiunea nucleului ESP32 și a bibliotecii ReadyMail folosite, pentru ca rezultatele să poată fi reproduse.

Biblioteci: WiFi, WebServer, HTTPClient, WiFiClientSecure, BluetoothSerial, FS și LittleFS vin cu nucleul ESP32; nu instalați biblioteci cu nume asemănătoare destinate altor platforme. ReadyMail se instalează separat din Library Manager.

Configurarea locală a credențialelor

În fiecare director T1, T2, T4 și T5 copiați secrets.example.h sub numele secrets.h, în același director cu .ino. Completați SSID-ul și parola primite la laborator. La T4 completați și telefonul/cheia API, iar la T5 expeditorul, parola de aplicație și destinatarul. Fișierul secrets.h este exclus din Git prin .gitignore. Nu redenumiți exemplul: păstrați-l pentru ceilalți studenți. Taskul T3 nu necesită secrets.h.

Cum folosim codul comun

Fișierul lab_config.h există deja lângă sketch-urile T1, T2, T4 și T5. Când se deschide un .ino, Arduino IDE afișează și acest fișier într-un tab separat. Dacă lucrați numai după PDF, creați un tab nou cu numele lab_config.h și copiați codul de mai jos. Creați și secrets.h pornind de la exemplul din repository; headerul comun îl include.

conectareWiFi() pornește placa în mod stație, încearcă asocierea și așteaptă cel mult 20 de secunde. La succes afișează adresa IP. La eșec întoarce false, astfel încât restul programului

poate opri operația curentă. sincronizareOra() obține timpul UTC prin NTP și are, de asemenea, un termen-limită; se folosește la e-mail și la validarea certificatelor.

```
#pragma once
#include <Arduino.h>
#include <WiFi.h>
#include <time.h>

#include "secrets.h"
const char* GRUPA = "G01";  // Fiecare echipa modifica.

bool conectareWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Conectare Wi-Fi");
  uint32_t t0 = millis();
  while (WiFi.status() != WL_CONNECTED && millis()-t0 < 20000) {
    delay(250);
    Serial.print('.');
  }
  Serial.println();
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Conectare esuata. Verificati reteaua si parola.");
    return false;
  }

  Serial.print("IP: "); Serial.println(WiFi.localIP());
  return true;
}

bool sincronizareOra() {
  configTime(0, 0, "pool.ntp.org", "time.nist.gov");
  uint32_t t0 = millis();
  while (time(nullptr) < 1700000000 && millis()-t0 < 20000)
    delay(250);
  return time(nullptr) >= 1700000000;
}
```

Exemplu de secrets.h pentru configurarea locală

Copiați liniile de mai jos într-un tab secrets.h; exemplele specifice fiecărui task sunt în repository. Valorile sunt substituenți, nu credențiale funcționale.

```
#pragma once
// Copiati ca secrets.h si completati local. Nu publicati acest fisier.
const char* WIFI_SSID = "SSID_LABORATOR";
const char* WIFI_PASS = "PAROLA_WIFI";
```

Organizarea lucrului și a rezultatelor

Fiecare echipă păstrează un jurnal cu: identificatorul grupei, placa, versiunile software, IP-ul primit, probele efectuate și erorile întâlnite. Parolele, cheile API și parola de aplicație nu se includ în capturile predate. Nu se trimit notificări repetate din loop(); în exemple, WhatsApp și e-mailul se declanșează manual.

Buget orientativ: pregătire 10 min; T1 10 min; T2 25 min; T3 20 min; T4 20 min; T5 30 min; concluzii 5 min. Activarea conturilor și instalarea extensiilor pot necesita timp suplimentar.

1. Task 1 · Testarea conexiunii Wi-Fi

Scop și explicație

La sfârșitul sarcinii, placa trebuie să fie asociată cu routerul laboratorului și să afișeze parametrii rețelei. ESP32 funcționează ca stație: folosește un punct de acces existent, nu creează o rețea nouă. SSID identifică rețeaua, iar parola permite autentificarea. Routerul atribuie de regulă configurația IPv4 prin DHCP.

Adresa IP identifică interfața în rețeaua curentă. Masca delimitează partea de rețea, gateway-ul este calea către alte rețele, iar DNS transformă numele serverelor în adrese. MAC este identificatorul interfeței la nivel de legătură. RSSI exprimă puterea semnalului recepționat, în dBm: în condiții comparabile, −45 dBm indică un semnal mai puternic decât −75 dBm. RSSI nu măsoară viteza conexiunii.

1 · Conectare la rețeaua locală



Routerul atribuie adrese prin DHCP. Conectarea la Wi-Fi nu garantează accesul la Internet.

Pașii de lucru

1. Deschideți T1_WiFi.ino. Verificați configurația: WIFI_SSID și WIFI_PASS în secrets.h, folosind datele comunicate de cadrul didactic. Dacă routerul a fost reconfigurat, folosiți datele comunicate în sală.

2. Încărcați programul. Deschideți Serial Monitor la 115200 baud. Notați IP, MAC, mască, gateway, DNS și canal.

3. Urmăriți minimum cinci valori RSSI. Deplasați ușor placa sau schimbați orientarea și comparați valorile; nu deduceți o distanță exactă dintr-o singură măsurare.

4. Comparați adresa IP cu cea a calculatorului conectat la aceeași rețea. Pe Windows, comanda ipconfig se introduce în Command Prompt sau PowerShell, nu în Serial Monitor.

5. Dacă doriți, testați ping către IP-ul plăcii. Un răspuns confirmă accesibilitatea IP în acel moment; lipsa răspunsului nu dovedește singură că asocierea Wi-Fi a eșuat.

Program complet · T1_WiFi.ino

```
#include "lab_config.h"

void setup() {
  Serial.begin(115200);
  delay(500);
  if (!conectareWiFi()) return;
  Serial.print("SSID: "); Serial.println(WiFi.SSID());
  Serial.print("MAC: "); Serial.println(WiFi.macAddress());
  Serial.print("Masca: "); Serial.println(WiFi.subnetMask());
  Serial.print("Gateway: "); Serial.println(WiFi.gatewayIP());
  Serial.print("DNS: "); Serial.println(WiFi.dnsIP());
  Serial.print("Canal: "); Serial.println(WiFi.channel());
}

void loop() {
  static uint32_t ultima = 0;
  if (millis()-ultima >= 3000) {
    ultima = millis();
    if (WiFi.status() == WL_CONNECTED)
      Serial.printf("RSSI: %d dBm\n", WiFi.RSSI());
    else
      Serial.println("Wi-Fi deconectat; apasati EN pentru retestare.");

  }
}
```

Interpretarea programului și validarea

setup() rulează o dată după resetare și afișează configurația. loop() verifică timpul scurs cu millis(), fără o pauză de trei secunde care să blocheze întreaga buclă. RSSI se citește numai cât timp placa este conectată.

Rezultat așteptat: o adresă IPv4 validă în rețeaua laboratorului și valori RSSI periodice. Adresele sunt alocate dinamic; nu copiați IP-ul unei alte grupe. Exemplu ilustrativ de jurnal: „IP: 192.168.x.y”, apoi „RSSI: -58 dBm”; valorile reale vor fi diferite.

Dacă nu merge: verificați caracterele SSID/parolă, banda de 2,4 GHz, alimentarea USB și apropierea de router. O rețea cu portal de autentificare web nu este echivalentă cu o rețea WPA configurată doar prin SSID și parolă. După corectare, reîncărcați sau apăsați EN.

De predat: o captură cu parametrii rețelei, cinci valori RSSI și două propoziții despre diferența dintre „conectat la Wi-Fi” și „are acces la Internet”. Referință API: S6.

2. Task 2 · Server web pentru LED-ul de pe GPIO 2

Scop și adaptarea montajului

Browserul va comanda aprinderea și stingerea LED-ului existent pe placă. Nu se conectează componente externe. Tutorialul indicat folosește două ieșiri, GPIO 26 și GPIO 27; adaptarea solicitată este const int output26 = 2;. În programul de laborator de mai jos această modificare este deja aplicată, iar canalul GPIO 27 a fost eliminat, împreună cu butoanele sale. [S1]

Numele variabilei output26 este păstrat pentru a face vizibilă corespondența cu indicația inițială; valoarea variabilei, 2, stabilește pinul efectiv. Interfața web și mesajele de diagnostic indică GPIO 2. LED_ACTIV_HIGH se setează false numai dacă LED-ul plăcii respective este activ pe nivel LOW. Unele plăci au alt LED sau nu au LED utilizabil pe GPIO 2; verificați modelul, fără a presupune că LED-ul de alimentare poate fi comandat.

2 · Comanda LED-ului din browser



POST /2/on → LED aprins • POST /2/off → LED stins • Nu se montează LED-uri externe.

Ce se întâmplă la apăsarea unui buton

ESP32 este server HTTP, iar browserul este client. Portul 80 identifică serviciul HTTP. Browserul cere pagina principală prin GET /. Pagina conține două formulare: apăsarea butonului trimite POST /2/on sau POST /2/off. Serverul schimbă ieșirea și răspunde cu o redirecționare 303 către pagina principală. Browserul face un nou GET și afișează starea memorată în program.

Acesta este un exemplu propriu bazat pe biblioteca WebServer, pentru a separa clar rutele de codul HTML. Articolul de referință procesează manual cererile prin WiFiServer. Principiul client–server și comanda ieșirii sunt aceleași; nu combinați fragmente din cele două implementări.

Execuție

1. Deschideți T2_Server_LED.ino și verificați lab_config.h. Încărcați programul, apoi deschideți Serial Monitor la 115200 baud.

2. Notați IP-ul primit. Pe telefon sau calculator conectați-vă la aceeași rețea locală și deschideți explicit http://IP_ESP32, înlocuind IP_ESP32 cu adresa afișată. Nu folosiți https://.

3. Apăsați PORNESTE, apoi OPRESTE. Urmăriți simultan LED-ul, starea din pagină și jurnalul serial.

4. Repetați dintr-un al doilea browser. Reîmprospătați prima pagină pentru a vedea starea modificată de celălalt client; exemplul nu trimite automat actualizări către toate paginile

deschise.

Program complet · T2_Server_LED.ino

```
#include "lab_config.h"
#include <WebServer.h>

const int output26 = 2;  // Adaptarea ceruta: LED-ul de pe placa.
const bool LED_ACTIV_HIGH = true;
bool ledPornit = false;
WebServer server(80);

void seteazaLED(bool pornit) {
  ledPornit = pornit;
  digitalWrite(output26, (pornit == LED_ACTIV_HIGH) ? HIGH : LOW);
  Serial.printf("GPIO 2: %s\n", pornit ? "ON" : "OFF");
}

String pagina() {
  String h = "<!doctype html><html lang='ro'><meta charset='utf-8'>";
  h += "<meta name='viewport' content='width=device-width,initial-scale=1'>";
  h += "<title>ESP32 - laborator 2</title>";
  h += "<style>body{font:22px Arial;text-align:center;margin:40px}";
  h += "button{font-size:24px;padding:18px;margin:12px}</style>";
  h += "<h1>ESP32 - "; h += GRUPA; h += "</h1>";
  h += "<p>LED incorporat: GPIO 2</p><p>Stare: <b>";

  h += ledPornit ? "ON" : "OFF";
  h += "</b></p><form action='/2/on' method='post'>";
  h += "<button>PORNESTE</button></form>";
  h += "<form action='/2/off' method='post'>";
  h += "<button>OPRESTE</button></form></html>";
  return h;
}

void redirectioneaza() {
  server.sendHeader("Location", "/");
  server.send(303, "text/plain", "Revenire la pagina principala");
}

void setup() {
  Serial.begin(115200);
  pinMode(output26, OUTPUT);
  seteazaLED(false);
  if (!conectareWiFi()) return;
  server.on("/", HTTP_GET, []() {
    server.sendHeader("Cache-Control", "no-store");
    server.send(200, "text/html; charset=utf-8", pagina());
  });

  server.on("/2/on", HTTP_POST, []() {
    seteazaLED(true); redirectioneaza();
  });
  server.on("/2/off", HTTP_POST, []() {
    seteazaLED(false); redirectioneaza();
  });
  server.onNotFound([]() { server.send(404, "text/plain", "Ruta inexistenta"); });
  server.begin();
  Serial.print("Deschideti http://"); Serial.println(WiFi.localIP());
}

void loop() {
  server.handleClient();
}
```

Explicarea funcțiilor

• seteazaLED() păstrează starea logică și calculează nivelul electric în funcție de polaritatea LED-ului. Variabila ledPornit descrie comanda; nu este o măsurare optică a aprinderii.

• pagina() construiește un șir HTML. HTML definește elementele paginii, iar CSS stabilește dimensiunile și aspectul butoanelor. Metadatele viewport adaptează afișarea la telefon.

• server.on() asociază o rută și o metodă HTTP unei funcții. Funcțiile scurte scrise între [] și acolade sunt expresii lambda C++.

• server.handleClient() trebuie apelată repetat în loop(). O întârziere mare introdusă aici încetinește răspunsul interfeței.

• Răspunsul 200 livrează pagina; 303 cere reîncărcarea prin GET; 404 indică o rută inexistentă. Cache-Control: no-store reduce riscul afișării unei pagini vechi din cache.

Probe obligatorii și depanare

Efectuați cinci cicluri ON/OFF. Deschideți /ruta-inexistenta și observați răspunsul. Apăsați EN și observați că LED-ul revine la OFF; programul nu salvează starea în memoria nevolatilă.

Dacă browserul nu ajunge la placă, verificați IP-ul după ultima resetare, prefixul http:// și izolarea clienților în router. Rețelele „Guest” pot împiedica comunicarea între dispozitive. Pe telefon verificați că cererea este trimisă prin Wi-Fi, nu printr-o rută alternativă VPN/date mobile. Dacă pagina merge, dar LED-ul nu se schimbă, verificați GPIO și polaritatea LED-ului.

De predat: captura paginii, două fotografii cu LED-ul în stări diferite și explicația de ce schimbarea numelui variabilei nu schimbă pinul, dar schimbarea valorii sale îl schimbă. Extensie: adăugați un contor al comenzilor primite și afișați-l pe pagină.

3. Task 3 · Bluetooth Classic și terminal serial

Scop și principiu

Realizați o punte bidirecțională între Serial Monitor și un terminal Bluetooth de pe telefon. Extensia laboratorului interpretează trei caractere primite prin Bluetooth pentru a controla LED-ul: 1, 0 și ?. Se folosește profilul SPP al Bluetooth Classic, nu BLE; un scaner BLE nu este clientul potrivit pentru acest program. Exemplul de bază din Arduino IDE este File → Examples → BluetoothSerial → SerialToSerialBT. [S2, S7]

3 · Două interfețe seriale și o comandă locală



Caracterul 1 aprinde LED-ul, 0 îl stinge, iar ? solicită starea. Routerul nu este necesar.

Asociere și testare

1. Folosiți o placă ESP32 clasică. Deschideți T3_Bluetooth.ino și schimbați NUME_BT în ESP32_L2_Gxx, cu identificatorul propriei echipe. Nu lăsați toate plăcile cu același nume.

2. Instalați pe Android un terminal SPP, de exemplu Serial Bluetooth Terminal, indicat în tutorialul S2. Acordați permisiunile Bluetooth/Nearby devices cerute de sistem.

3. Încărcați codul. În setările Bluetooth ale telefonului căutați dispozitivul, efectuați asocierea și confirmați solicitarea afișată, dacă există. Nu presupuneți automat un PIN 1234; procedura depinde de versiunile software.

4. În aplicația terminal selectați dispozitivul din categoria Bluetooth Classic și conectați-vă. Asocierea în setările Android și conexiunea SPP în aplicație sunt două operații diferite.

5. Trimiteți „Salut din PC” din Serial Monitor. Textul trebuie să apară pe telefon. Trimiteți „Salut din telefon” în sens invers și urmăriți monitorul serial.

6. Trimiteți separat 1, 0 și ?. Prima comandă aprinde LED-ul, a doua îl stinge, iar a treia întoarce starea. Terminatoarele CR/LF sunt tolerate și nu schimbă ieșirea.

Telefon iPhone: aplicațiile iOS obișnuite nu oferă acces generic la SPP în același mod ca Android. Pentru acest task utilizați un Android sau un client SPP compatibil de pe PC; o versiune BLE ar fi o aplicație diferită.

Program complet · T3_Bluetooth.ino

```
#include <Arduino.h>
#include "BluetoothSerial.h"
#if !defined(CONFIG_BT_SPP_ENABLED)
#error "Acest task necesita ESP32 clasic cu Bluetooth SPP."
#endif

BluetoothSerial BT;
const char* NUME_BT = "ESP32_L2_G01";
const int LED_PIN = 2;
const bool LED_ACTIV_HIGH = true;
bool ledPornit = false;

void comanda(char c) {
  if (c != '0' && c != '1' && c != '?') return;
  if (c == '0' || c == '1') {
    ledPornit = (c == '1');
    digitalWrite(LED_PIN, (ledPornit == LED_ACTIV_HIGH) ? HIGH : LOW);
  }
  String raspuns = String("LED=") + (ledPornit ? "ON" : "OFF");
  BT.println(raspuns);
  Serial.println(raspuns);
}

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LED_ACTIV_HIGH ? LOW : HIGH);
  if (!BT.begin(NUME_BT)) {
    Serial.println("Pornirea Bluetooth a esuat.");
    return;
  }
  Serial.print("Asociati telefonul cu: "); Serial.println(NUME_BT);
  Serial.println("Comenzi telefon: 1=ON, 0=OFF, ?=stare.");
}

void loop() {
  // Punte USB -> Bluetooth: fiecare octet este transmis telefonului.
  while (Serial.available()) BT.write(Serial.read());
  // Bluetooth -> USB; in plus interpretam cele trei comenzi.
  while (BT.available()) {
    char c = (char)BT.read();
    Serial.write(c);
    comanda(c);
  }

  delay(5);
}
```

Cum se interpretează datele

Serial și BT sunt două interfețe diferite. available() arată dacă există octeți recepționați, read() extrage un octet, iar write() îl transmite. În sens PC → telefon se transmite textul fără interpretarea comenzilor. În sens telefon → placă, fiecare caracter ajunge și pe USB, iar 1, 0 și ? sunt interpretate suplimentar.

Comanda este bazată pe caractere, nu pe mesaje complete. De exemplu, textul „grupa1” trimis de pe telefon conține și o comandă de aprindere. În acest laborator trimiteți comenzile separat. O extensie utilă este acumularea unei linii până la Enter și recunoașterea cuvintelor ON, OFF și STATUS.

Validare: demonstrați transmisia în ambele sensuri și cele trei comenzi. Deconectați terminalul și reconectați-l. Observați că nu a fost necesar routerul sau accesul la Internet.

Depanare: dacă dispozitivul lipsește, verificați numele și modelul plăcii. Dacă este asociat, dar nu primește date, conectați terminalul la serviciul SPP și închideți un alt client deja conectat. Dacă după schimbarea programului asocierea eșuează, eliminați asocierea veche din telefon și repetați procedura.

De predat: capturi din ambele terminale și răspunsul la întrebarea: de ce 115200 baud din Serial Monitor nu reprezintă viteza radio Bluetooth?

4. Task 4 · Mesaj WhatsApp de la ESP32

Scop, dependențe și disponibilitate

ESP32 va iniția o cerere HTTPS către CallMeBot, care livrează mesajul către propriul cont WhatsApp. Telefonul nu este conectat direct la ESP32, iar placa nu implementează protocolul WhatsApp. Sunt necesare conexiune la Internet, număr de telefon în format internațional și cheie API activată pentru acel număr. [S3, S5]

Situația verificată la 06.10.2026: pagina oficială CallMeBot anunță că botul este plin și că numărul de activare va reapărea când vor exista locuri. Nu folosiți un număr vechi preluat din capturi sau tutoriale. Cu o cheie deja obținută se poate încerca proba reală, fără garanția disponibilității serviciului. Fără cheie, efectuați varianta SIMULARE și consemnați că notificarea nu a fost livrată. [S5]

4 · Notificare printr-un serviciu intermediar



SIMULARE = true: se verifică local construirea cererii; nu se efectuează trimiterea.

Activare, când serviciul permite

1. Deschideți pagina oficială S5 și folosiți numai numărul botului afișat acolo. Salvați contactul pe telefon.

2. Trimiteți din WhatsApp către acel contact mesajul de autorizare: „I allow callmebot to send me messages”. Așteptați confirmarea cu cheia API. Această acțiune o efectuează studentul de pe propriul telefon.

3. Introduceți în secrets.h TELEFON, de exemplu +407xxxxxxxx, fără spații, și API_KEY. Cheia nu este parola WhatsApp.

4. Pentru trimitere reală schimbați SIMULARE în false. Verificați rețeaua în lab_config.h. Încărcați codul și trimiteți W din Serial Monitor.

5. Notați codul HTTP și verificați efectiv primirea pe telefon. Nu repetați rapid cererea: codul impune 60 de secunde între încercări.

Formatul cererii și codificarea textului

O cerere conține gazda, resursa și parametrii phone, text și apikey. Codul de aici folosește o cerere GET către endpointul documentat de serviciu. În URL, caracterele care pot avea rol de separator trebuie codificate. De exemplu: spațiu → %20, plus → %2B, & → %26. codificaURL() prelucrează octeții UTF-8, astfel încât și un mesaj cu diacritice poate fi codificat.

Nu confundați două rezultate: codul 200 indică un răspuns HTTP reușit, iar confirmarea finală a taskului este mesajul primit în aplicație. O limitare a serviciului sau o problemă a contului poate interveni după acceptarea cererii.

Program complet · T4_WhatsApp.ino

```
#include "lab_config.h"
#include <HTTPClient.h>
#include <WiFiClientSecure.h>

// true: exercitiu local, NU se trimite un mesaj WhatsApp.
const bool SIMULARE = true;
// Optiunea tutorialului, numai pentru demonstratie controlata.
const bool TLS_DEMO_FARA_VALIDARE = true;
const char* CA_PEM = "";  // Certificatul CA PEM, pentru validare TLS.

String codificaURL(const String& s) {
  const char hex[] = "0123456789ABCDEF";
  String out;
  for (size_t i=0; i<s.length(); i++) {
    uint8_t c = (uint8_t)s[i];
    if ((c>='A' && c<='Z') || (c>='a' && c<='z') ||
        (c>='0' && c<='9') || c=='-' || c=='_' || c=='.' || c=='~')
      out += (char)c;
    else {
      out += '%'; out += hex[c>>4]; out += hex[c & 15];
    }
  }

  return out;
}

void trimite() {
  String mesaj = String("Laborator 2 - ") + GRUPA + ": Salut de la ESP32!";
  if (SIMULARE) {
    Serial.println("SIMULARE: niciun mesaj nu a fost trimis.");
    Serial.println("Gazda: api.callmebot.com, resursa: /whatsapp.php");
    Serial.print("text="); Serial.println(codificaURL(mesaj));
    Serial.println("phone si apikey: ascunse. Configurati pentru test real.");
    return;
  }
  if (String(API_KEY)=="COMPLETEAZA_CHEIA" || String(TELEFON).indexOf('X')>=0) {
    Serial.println("Completati telefonul si cheia API."); return;
  }
  if (WiFi.status()!=WL_CONNECTED) {
    Serial.println("Nu exista conexiune Wi-Fi."); return;
  }
  WiFiClientSecure tls;
  if (TLS_DEMO_FARA_VALIDARE) tls.setInsecure();
  else {
    if (!strlen(CA_PEM) || !sincronizareOra()) {

      Serial.println("Validarea TLS necesita CA PEM si ora corecta."); return;
    }
    tls.setCACert(CA_PEM);
  }
  String url = "https://api.callmebot.com/whatsapp.php?phone=";
  url += codificaURL(TELEFON);
  url += "&text=" + codificaURL(mesaj);
  url += "&apikey=" + codificaURL(API_KEY);
  HTTPClient http;
  http.setTimeout(15000);
  if (!http.begin(tls, url)) {
    Serial.println("Initializare HTTP esuata."); return;
  }
  int cod = http.GET();
  Serial.printf("Rezultat HTTP: %d\n", cod);
```

```
  if (cod==200)
    Serial.println("Cerere acceptata HTTP; verificati primirea in WhatsApp.");
  else
    Serial.println("Cerere nereusita; verificati reteaua, cheia si serviciul.");
  http.end();
}

void setup() {
  Serial.begin(115200);
  if (!SIMULARE) conectareWiFi();
  Serial.println("Trimiteti W din Serial Monitor. Pauza minima: 60 s.");
}

void loop() {
  static bool trimis = false;
  static uint32_t ultima = 0;
  if (Serial.available()) {
    char c = Serial.read();
    if (c!='W' && c!='w') return;
    if (trimis && millis()-ultima < 60000) {
      Serial.println("Asteptati 60 s intre incercari."); return;
    }
    ultima=millis(); trimis=true; trimite();
  }
}
```

Varianta locală de rezervă

Cu SIMULARE = true, programul nu cere Wi-Fi și nu contactează CallMeBot. Trimiteți W și observați mesajul codificat. Schimbați mesajul în „Grupa G01: temperatura = 23 & stare = OK”, apoi verificați apariția %20, %3D și %26. Testați codificaURL() și cu șirul „+40712345678”; plusul trebuie să devină %2B.

Această variantă verifică construirea parametrilor și declanșarea manuală. Nu testează DNS, TLS, accesul la API sau livrarea WhatsApp. În raport se scrie explicit „test local, fără livrare”, dacă nu s-a efectuat proba reală.

TLS și tratarea erorilor

TLS_DEMO_FARA_VALIDARE = true activează setInsecure(), pentru demonstrația simplificată din laborator. Conexiunea este criptată, dar identitatea serverului nu este verificată. Utilizați numai un cont de test și nu predați cod cu cheia reală. Pentru validare TLS, setați opțiunea false, introduceți certificatul CA PEM corespunzător lanțului serverului și asigurați sincronizarea orei; CA_PEM gol oprește intenționat operația.

Un cod negativ returnat de HTTPClient indică o problemă locală de transport, conexiune sau TLS. Pentru răspunsuri HTTP de eroare verificați cheia, numărul și starea serviciului; dacă apar limitări de frecvență, opriți repetarea și consultați instrucțiunile furnizorului. Nu publicați URL-ul complet, deoarece conține cheia.

De predat: mesajul primit, cu numărul/cheia ascunse, sau captura simulării și menționarea limitării. Explicați de ce o notificare Internet nu funcționează doar pentru că taskul 1 a afișat o adresă IP.

5. Task 5 · E-mail prin SMTP: text, HTML și atașamente

Scop și arhitectură

Realizați trei probe cu același program: mesaj text, mesaj HTML și mesaj HTML cu două fișiere atașate. ESP32 este client SMTP și predă mesajul unui server de e-mail. Serverul efectuează livrarea către destinatar. SMTP este folosit pentru expediere; nu este protocolul prin care browserul afișează pagina din taskul 2.

Articolul de referință a fost actualizat la 25 mai 2026 și utilizează ReadyMail. Vechea ESP-Mail-Client este declarată neîntreținută de autor. Materialul folosește ReadyMail, fără amestecarea tipurilor și funcțiilor din cele două biblioteci. [S4, S8, S12]

5 · E-mail și fișiere din memoria plăcii



Parola de aplicație autorizează expeditorul. Fiecare comandă E inițiază o singură trimitere.

Contul expeditor și biblioteca

1. Folosiți un cont de laborator pentru expediere. În contul Google activați verificarea în doi pași și generați o parolă de aplicație pentru ESP32. Instrucțiunea oficială și condițiile sunt în S9.

2. Dacă opțiunea App passwords lipsește, contul poate avea restricții de organizație, Advanced Protection sau o configurație care nu permite această metodă. Folosiți un cont eligibil ori date SMTP aprobate de administrator; nu dezactivați politicile contului.

3. În Arduino IDE deschideți Library Manager, căutați ReadyMail și instalați biblioteca de Mobizt. Exemplul a fost confruntat cu API-ul 0.4.2; notați versiunea instalată.

4. În secrets.h din directorul T5_Email completați EXPEDITOR, PAROLA_APP și DESTINATAR. PAROLA_APP este parola de aplicație, nu parola obișnuită de autentificare. Copiați-o fără spațiile de grupare afișate de interfață.

5. Pentru exemplul Gmail păstrați smtp.gmail.com și portul 465, cu TLS de la începutul conexiunii. Portul 587 folosește în mod obișnuit STARTTLS și nu se substituie pur și simplu în acest exemplu.

Etapele de funcționare

După conectarea Wi-Fi, programul așteaptă comanda E. Funcția trimiteEmail() verifică datele, sincronizează ora și, în modul cu atașamente, verifică sistemul de fișiere. Apoi deschide conexiunea SMTP, autentifică expeditorul, construiește antetele și conținutul, transmite mesajul și închide conexiunea.

Antetele From și To stabilesc adresele afișate, iar Subject conține grupa. text/plain păstrează textul ca atare; text/html permite marcaje precum titlu, paragraf și aldine. MIME descrie tipurile conținutului și ale fișierelor. Un PNG trebuie trimis ca image/png, iar fișierul TXT ca text/plain. Biblioteca se ocupă de împachetarea mesajului și de codificarea atașamentelor binare.

Program complet · T5_Email.ino

```
// Implementare didactica pe baza API-ului ReadyMail (Mobizt).
// Pentru licenta bibliotecii consultati LICENSE_ReadyMail.txt.
#include "lab_config.h"
#include <WiFiClientSecure.h>
#include <FS.h>
#include <LittleFS.h>
#define ENABLE_SMTP
#define ENABLE_DEBUG
#define ENABLE_FS
#include <ReadyMail.h>

const char* SMTP_HOST = "smtp.gmail.com";
const uint16_t SMTP_PORT = 465;
// 0=text simplu, 1=HTML, 2=HTML + TXT + PNG din LittleFS.
const int MOD_EMAIL = 0;
const bool TLS_DEMO_FARA_VALIDARE = true;
const char* CA_PEM = ""; // CA PEM pentru smtp.gmail.com in modul validat.
WiFiClientSecure tls;
SMTPClient smtp(tls);

void stareSMTP(SMTPStatus status) {
  Serial.println(status.text);

}

void fisierCitire(File& f, const char* nume,
                  readymail_file_operating_mode mod) {
  if (mod==readymail_file_mode_open_read) f=LittleFS.open(nume, "r");
}

void atasament(SMTPMessage& m, const char* nume,
               const char* mime, const char* cale) {
  Attachment a;
  a.filename=nume; a.name=nume; a.mime=mime;
  a.attach_file.callback=fisierCitire;
  a.attach_file.path=cale;
  m.attachments.add(a, attach_type_attachment);
}

void trimiteEmail() {
  if (String(PAROLA_APP)=="COMPLETEAZA_PAROLA_DE_APLICATIE") {
    Serial.println("Completati expeditorul, parola APP si destinatarul.");
    return;
  }
  if (WiFi.status()!=WL_CONNECTED) {

    Serial.println("Nu exista Wi-Fi."); return;
  }
  if (!sincronizareOra()) {
    Serial.println("NTP indisponibil. Verificati accesul la Internet."); return;
  }
  if (MOD_EMAIL==2) {
    // false: NU formata automat; trebuie pastrate fisierele incarcate.
    if (!LittleFS.begin(false)) {
      Serial.println("LittleFS indisponibil. Incarcati folderul data."); return;
    }
    if (!LittleFS.exists("/text_file.txt") || !LittleFS.exists("/image.png")) {
      Serial.println("Lipseste text_file.txt sau image.png din LittleFS."); return;
    }
  }
```

```
  if (TLS_DEMO_FARA_VALIDARE) tls.setInsecure();
  else {
    if (!strlen(CA_PEM)) { Serial.println("Lipseste CA PEM."); return; }
    tls.setCACert(CA_PEM);
  }
  smtp.connect(SMTP_HOST, SMTP_PORT, stareSMTP);
  if (!smtp.isConnected()) { tls.stop(); return; }
  smtp.authenticate(EXPEDITOR, PAROLA_APP, readymail_auth_password);

  if (!smtp.isAuthenticated()) { tls.stop(); return; }
  SMTPMessage m;
  m.headers.add(rfc822_from, String("ESP32 <")+EXPEDITOR+">");
  m.headers.add(rfc822_to, String("Student <")+DESTINATAR+">");
  m.headers.add(rfc822_subject, String("Laborator 2 - ")+GRUPA);
  String corp=String("Mesaj de laborator, grupa ")+GRUPA+". ESP32 functioneaza.";
  if (MOD_EMAIL==0) m.text.body(corp);
  else m.html.body(String("<html><body><h2>Raport ESP32</h2><p>")+
                   corp+"</p><p><b>Status: OK</b></p></body></html>");
  if (MOD_EMAIL==2) {
    atasament(m, "text_file.txt", "text/plain", "/text_file.txt");
    atasament(m, "image.png", "image/png", "/image.png");
  }
  m.timestamp=time(nullptr);
  bool ok=smtp.send(m);  // Mod sincron: asteapta rezultatul.
  Serial.println(ok ? "SMTP: trimitere reusita. Verificati Inbox/Spam."
                    : "SMTP: trimitere esuata. Consultati jurnalul.");
  tls.stop();
}

void setup() {
  Serial.begin(115200);

  conectareWiFi();
  Serial.println("Trimiteti E pentru UN email. Pauza minima: 60 s.");
}

void loop() {
  static uint32_t ultima=0;
  static bool incercat=false;
  if (Serial.available()) {
    char c=Serial.read();
    if (c!='E' && c!='e') return;
    if (incercat && millis()-ultima<60000) {
      Serial.println("Asteptati 60 s."); return;
    }
    ultima=millis(); incercat=true; trimiteEmail();
  }
}
```

Proba A · Mesaj text

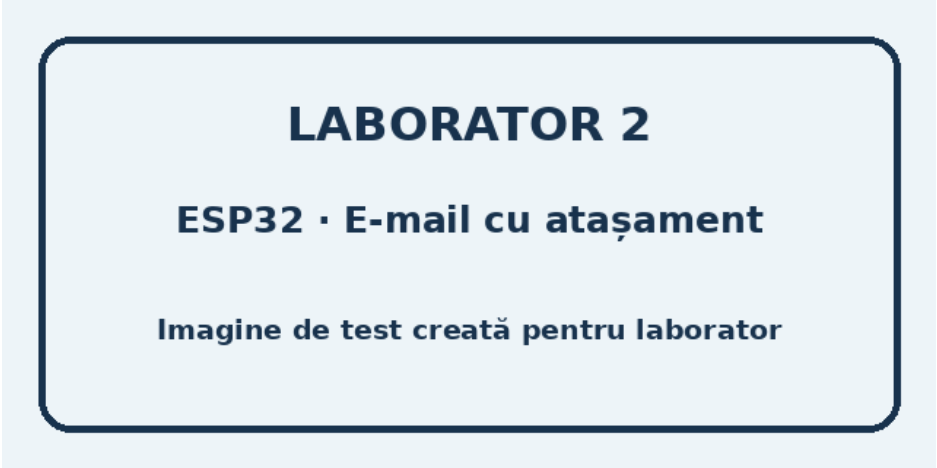
Setați MOD_EMAIL = 0, completați credențialele și încărcați. Deschideți Serial Monitor la 115200 baud și trimiteți E. Urmăriți conectarea, autentificarea și rezultatul trimiterii. Verificați Inbox și Spam. Subiectul trebuie să conțină grupa, iar corpul mesajului să fie text simplu. Nu este necesară încărcarea fișierelor LittleFS pentru această probă.

Proba B · Mesaj HTML

Setați MOD_EMAIL = 1, reîncărcați și trimiteți E. În mesaj trebuie să apară titlul „Raport ESP32” și textul „Status: OK” cu formatarea definită. Modificați titlul și adăugați un paragraf cu numele grupei. Clientul de e-mail poate filtra sau adapta stilurile HTML; un mesaj e-mail nu este o pagină web cu JavaScript executabil.

Proba C · Două atașamente din LittleFS

LittleFS este un sistem de fișiere în memoria flash a plăcii. Directorul data de pe calculator nu devine accesibil pe ESP32 doar prin Upload al sketch-ului: se efectuează separat încărcarea imaginii sistemului de fișiere. Pachetul include T5_Email/data/text_file.txt și T5_Email/data/image.png. Imaginea este originală și poate fi folosită în exercițiu.



1. Dacă extensia lipsește, descărcați fișierul .vsix din Releases al proiectului S10. Pe Windows copiați-l în C:\Users\NUME_UTILIZATOR\.arduinoIDE\plugins\; creați directorul dacă este necesar. Reporniți Arduino IDE. Pe Linux/macOS directorul este ~/.arduinoIDE/plugins/.

2. Deschideți T5_Email.ino din directorul său. În Sketch → Show Sketch Folder verificați existența directorului data și a celor două fișiere. Deschideți TXT-ul și modificați grupa; păstrați numele fișierelor.

3. Selectați o schemă de partiționare care include o zonă pentru sistemul de fișiere, potrivită memoriei plăcii. Pentru plăcile clasice de 4 MB este uzuală o schemă Default cu partiție de date. Nu schimbați schema între încărcarea sketch-ului și a fișierelor.

4. Setați MOD_EMAIL = 2 și încărcați sketch-ul. Închideți Serial Monitor înainte de încărcarea LittleFS, pentru a elibera portul serial.

5. Apăsați Ctrl+Shift+P și selectați „Upload LittleFS to Pico/ESP8266/ESP32”. Așteptați confirmarea succesului. Pe macOS folosiți Cmd+Shift+P. Dacă placa nu intră automat în bootloader, utilizați BOOT ca la încărcarea sketch-ului. [S10]

6. Deschideți din nou Serial Monitor, apăsați EN și trimiteți E. Deschideți mesajul și verificați ambele atașamente, nu doar prezența pictogramei de fișier.

Atenție la formatare: LittleFS.begin(false) nu șterge automat zona de date dacă montarea eșuează. Încărcarea unei imagini LittleFS înlocuiește conținutul acelei partiții; în acest laborator se folosește zona de test. Nu înlocuiți false cu true ca soluție rapidă după ce ați încărcat fișierele, deoarece o formatare ar elimina atașamentele.

Explicarea codului pentru atașamente

fisierCitire() este callback-ul prin care ReadyMail deschide un fișier la cerere. Programul are nevoie numai de operația de citire; fișierele au fost încărcate anterior. atasament() asociază numele vizibil, tipul MIME și calea din LittleFS. /image.png este calea pe placă, nu o cale Windows.

MOD_EMAIL selectează o singură probă pe compilare. Funcția smtp.send() este folosită în modul sincron: așteaptă încheierea operației și întoarce rezultatul. Pauza de minimum 60 de secunde din

loop() limitează apăsările repetate, dar nu constituie o garanție de livrare sau o cotă autorizată de furnizor.

TLS: exemplul include același mod didactic fără validarea certificatului ca la taskul 4. Pentru un sistem real utilizați CA_PEM corect, TLS_DEMO_FARA_VALIDARE = false și un cont configurat conform politicilor furnizorului. Nu distribuiți parola de aplicație împreună cu proiectul.

Depanare și criterii de reușită

• Eroare de autentificare: verificați adresa completă a expeditorului, parola de aplicație și eligibilitatea contului. Nu copiați parole reale în raport.

• Conexiune SMTP imposibilă: verificați DNS, accesul la Internet și dacă rețeaua permite portul 465. Conectarea la router nu este suficientă.

• NTP indisponibil: verificați accesul la serverele de timp. Programul oprește trimiterea după termenul-limită, în loc să aștepte nelimitat.

• LittleFS indisponibil sau fișiere lipsă: verificați încărcarea folderului data, schema partițiilor și numele exacte, inclusiv literele mari/mici.

• Mesaj trimis, dar absent din Inbox: verificați Spam, adresa destinatarului și întârzierile de livrare. Acceptarea SMTP nu dovedește că mesajul a fost citit.

De predat: câte o captură pentru mesajul text și HTML, plus captură cu cele două atașamente deschise. Precizați versiunea ReadyMail și de ce parola de aplicație este diferită de parola Wi-Fi.

6. Fișa de rezultate și întrebări de verificare

Completați pentru fiecare task: rezultat observat, dovada, eroarea întâlnită și corecția aplicată. Nu declarați reușita unei notificări doar din mesajul de diagnostic al plăcii.

Sarcină	Criteriu minim de validare	Date de consemnat
T1 · Wi-Fi	IP valid și jurnal RSSI	IP, mască, gateway, cinci RSSI
T2 · HTTP	Cinci cicluri ON/OFF pe GPIO 2	IP, capturi, reacția după reset
T3 · Bluetooth	Text în ambele sensuri și 1/0/?	Nume BT, comenzi, răspunsuri
T4 · WhatsApp	Mesaj primit sau simulare declarată	HTTP + primire / limitarea testului
T5 · E-mail	Text, HTML, TXT și PNG recepționate	Versiune, capturi, atașamente

1. Ce informație suplimentară oferă un IP valid față de simpla detectare a SSID-ului?

2. De ce serverul web poate funcționa fără Internet, în timp ce CallMeBot nu poate?

3. Care este diferența dintre starea logică ledPornit și nivelul electric pe GPIO?

4. Ce deosebește asocierea Bluetooth de deschiderea unei conexiuni SPP?

5. De ce se codifică semnul + într-un URL? Ce se întâmplă dacă textul conține &?

6. Ce diferență există între „cerere HTTP acceptată”, „mesaj livrat” și „mesaj citit”?

7. De ce fișierele pentru LittleFS se încarcă separat de program?

8. Ce protecție lipsește atunci când se folosește setInsecure(), chiar dacă transportul este TLS?

Extensie integratoare: adăugați în interfața web un buton de solicitare a unui raport prin e-mail. Transmiterea se face o singură dată pentru o acțiune explicită, cu limitarea frecvenței. Nu puneți operația de trimitere necondiționat în loop().

7. Surse și corespondența cu îndrumătorul inițial

Documentul de plecare este „Îndrumător laborator Sisteme dedicate pentru IoT – P1 – ESP32”, laboratorul 2, inclus în PDF-ul asamblat. S1–S4 sunt cele patru linkuri indicate acolo; taskul 1 trimitea la un fișier local, nu la un articol web. Celelalte surse verifică API-urile, activarea serviciilor și încărcarea fișierelor.

[S1] Random Nerd Tutorials · ESP32 Web Server – Arduino IDE. Referință pentru taskul 2; montajul cu două LED-uri este înlocuit aici cu GPIO 2 și un singur LED.
https://randomnerdtutorials.com/esp32-web-server-arduino-ide/

[S2] Random Nerd Tutorials · ESP32 Bluetooth Classic with Arduino IDE – Getting Started. Referință pentru puntea serială și terminalul Android.
https://randomnerdtutorials.com/esp32-bluetooth-classic-arduino-ide/

[S3] Random Nerd Tutorials · ESP32: Send Messages to WhatsApp. Referință pentru integrarea cu serviciul CallMeBot. https://randomnerdtutorials.com/esp32-send-messages-whatsapp/

[S4] Random Nerd Tutorials · ESP32 Send Emails using an SMTP Server: HTML, Text, and Attachments. Versiunea consultată indică actualizarea din 25.05.2026 și ReadyMail.
https://randomnerdtutorials.com/esp32-send-email-smtp-server-arduino-ide/

[S5] CallMeBot · Free API to Send WhatsApp Messages. Sursa pentru activare, endpoint și disponibilitatea înscrierilor. https://www.callmebot.com/blog/free-api-whatsapp-messages/

[S6] Espressif · Arduino ESP32 Wi-Fi API.
https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html

[S7] Espressif · Arduino ESP32 Bluetooth.
https://docs.espressif.com/projects/arduino-esp32/en/latest/api/bluetooth.html

[S8] Mobizt · ReadyMail; exemplele SimpleText și Attachment, API-ul SMTPClient. Adaptările acestui material adaugă configurarea laboratorului, declanșarea manuală, termene-limită și verificarea fișierelor. Licența autorului consultată este CC BY-NC 4.0; o copie este în arhivă.
https://github.com/mobizt/ReadyMail

[S9] Google Account Help · Sign in with app passwords.
https://support.google.com/accounts/answer/185833

[S10] Earle F. Philhower · Arduino LittleFS Upload, instalare și utilizare.
https://github.com/earlephilhower/arduino-littlefs-upload

[S11] Espressif · Instalarea Arduino ESP32.
https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html

[S12] Mobizt · ESP-Mail-Client, notificare de depreciere și recomandarea ReadyMail.
https://github.com/mobizt/ESP-Mail-Client

Surse consultate la 06.10.2026. Imaginile din acest îndrumător sunt scheme explicative proprii și o imagine de test; nu sunt capturi de ecran de pe dispozitivele studenților. Capturile și ilustrațiile originale ale tutorialelor pot fi consultate la S1–S4.